Project 2: The Delphian Succinct Argument of Knowledge

1 Introduction

In this project, you'll be using the arithmetic and polynomial libraries you wrote in Project 1 to build the first major cryptographic protocol milestone of the course, an interactive succinct argument of knowledge for the satisfiability of rank-one constraint systems. This protocol is called Delphian.

1.1 Provided Code

We have provided method stubs for the functions you are expected to implement. We have also provided starter code for a few key pieces. You need not understand every single detail of how this code works. In the starter code, please read the documentation in `src/ip.rs`, and `src/combined.rs` for information on the `InteractiveProof` trait, the `Comms` struct, and the `CombinedMLE` struct. In addition, your implementation of Delphian will use `src/sparsemat.rs`. This project will make use of `async` functions in order to simulate interaction between the prover and the verifier. You won't need to know these at a very deep level, but Chapter 17 of the Rust Book is a good introduction.

To set up this project, put the project2 starter code in the same parent directory as your project 1 code (that is the directory structure should look like `my_projects/p1` and `my_projects/p2`). Additionally, two changes will have to be made to your project 1 code in order for the project 2 code to compile. First, a `num_vars` method will need to be added to `Multilinear`. It should just return `self.n_vars`. Secondly, in `p1/src/lib.rs` `Random` needs to be added to the long list of supertraits of `Field`. Concretely, you should add `+ Random` to line 56 of that file. These changes have also been made to the P1 starter code on github if you would rather use that.

Your implementations will go in `src/quokka.rs`, `src/sumcheck.rs`, and `src/delphian.rs`. Each of these requires implementing a `prover` and `verifier` function. Quokka additionally requires implementing a function called `commit`. In terms of implementation order, we recommend implementing Quokka and Sumcheck first (in any order) followed by Delphian.

Rank-one constraint systems (R1CS). The protocol Delphian proves the R1CS relation we discussed in class. We implement R1CS as a struct in `delphian.rs`. The `Statement` struct contains the matrices A, B, C and the public input $\vec{x}$, represented in sparse form (see `sparsemat.rs` for the sparse matrix implementation). The `Witness` struct is for the witness $\vec{w}$. Be mindful of the length restrictions in the constructors for these structs—for simplicity we restrict both $\vec{x}$ and $\vec{w}$ to be the same size, and this size must be an *even* power of two. This simplifies certain aspects of the implementations of the protocols for this week. It is possible, and not particularly difficult, to remove this restriction either by tweaking the protocol slightly or by padding $\vec{x}$ and $\vec{w}$, but you don't need to do it.

$\mathsf{Sumcheck.Prover}(g, H)$		$\mathsf{Sumcheck.Verifier}(H)$
Compute $g_1(X_1)$ as $\sum_{x_2,\ldots,x_v \in \{0,1\}} g(X_1, x_2, \ldots, x_v)$		
	$\xrightarrow{\quad g_1 \quad}$	
		Check $H = g_1(0) + g_1(1)$ $r_1 \leftarrow\$ \mathbb{F}$
	$\xleftarrow{\quad r_1 \quad}$	
..... In round j:		
Compute $g_j(X_j)$ as $\sum_{x_{j+1},\ldots,x_v \in \{0,1\}} g(r_1, \ldots, r_{j-1}, X_j, x_{j+1}, \ldots, x_v)$		
	$\xrightarrow{\quad g_j \quad}$	
		Check $g_{j-1}(r_{j-1}) = g_j(0) + g_j(1)$ $r_j \leftarrow\$ \mathbb{F}$
	$\xleftarrow{\quad r_j \quad}$	
..... Final round:		
$g_v \leftarrow g(r_1, \ldots, r_{v-1}, X_v)$		
	$\xrightarrow{\quad g_v \quad}$	
		Check $g_{v-1}(r_{v-1}) = g_v(0) + g_v(1)$ $r_v \leftarrow\$ \mathbb{F}$; $H' \leftarrow g_v(r_v)$ If all checks pass: Return$(r_1, \ldots, r_v), H'$ Else return false

Figure 1: The sumcheck protocol rewritten as a reduction. The input claim is $\sum_{x_1,\ldots,x_n \in \{0,1\}} g(x_1, \ldots, x_n) = H$ and the output claim is that, for vector $r = (r_1, \ldots, r_n) \in \mathbb{F}^n$ composed of the verifier's random challenges and field element H', $g(r) = H'$.

2 The Sumcheck Protocol

Your first task in this project will be to implement a slight tweak to the sumcheck protocol we discussed in lecture. In lecture we described the sumcheck protocol as an interactive proof for the summation of a multivariate polynomial g in n variables over the "unit hypercube"—i.e., the set of all vectors in $\mathbb{F}^n$ with binary entries.

The Delphian succinct argument of knowledge (§ 3) you will implement uses sumcheck as a subprotocol. However, for Delphian it is convenient to describe sumcheck slightly differently: namely, instead of a standard interactive protocol, where the verifier outputs either zero or one, we think of sumcheck as a protocol that *reduces* one claim to another. The role of the prover and verifier are slightly different: intuitively, the prover's job is to convince the verifier that an input claim is, with overwhelming probability, equivalent to a related claim that the prover and verifier jointly construct during the protocol. We think of this constructed claim as being the verifier's output in the protocol.

We give this slightly different sumcheck in Figure 1. **This is the version you will implement in the project.** The initial claim is that for a multivariate polynomial g in n variables and field element H, $\sum_{x_1,\ldots,x_n \in \{0,1\}} g(x_1, \ldots, x_n) = H$, and the ending claim is that, for field element H' and polynomial g and random point $r = (r_1, \ldots, r_n) \in \mathbb{F}^n$, $g(r) = H'$.

This framing of sumcheck as a reduction is convenient because it allows using sumcheck in settings where the verifier may not be able to evaluate the polynomial g directly, either because it is too large for an efficient verifier to materialize or because revealing it to the verifier would leak information. Indeed, in Delphian, sumcheck is applied to a polynomial that is as large as the R1CS instance; thus the verifier cannot directly evaluate it if the protocol is to be succinct.

An important subtlety of this framing is that we must treat the verifier's output as known to both parties. (In Delphian, the prover would be unable to finish the protocol without knowledge of all the r_is, *including* the one generated by the verifier in the last step.)

2.1 Quokka: A Multilinear Polynomial Commitment

The next task is to implement the polynomial commitment scheme that will be used in the Delphian protocol. We will call our scheme Quokka. We give the algorithms for Quokka in Figure 2. The Quokka scheme has several desirable properties for us: its commitments are sublinear in the size of the committed polynomial, its opening protocol has both sublinear communication and verifier computation, and it allows committing to the evaluation form—rather than the coefficient form—of a multilinear extension. (Note that the ideas underlying Quokka are more general in that they allow building PCSs for univariate polynomials as well, but we will only use it for evaluation-form multilinear extensions.)

Intuition for Quokka**.** Let the number of evaluations be $n = 2^\ell$ and let n be a perfect square; call $m = \sqrt{n} = 2^{\ell/2}$. Let $p\colon \{0,1\}^\ell \to \mathbb{F}$ be a function, and $\tilde{p}$ be its multilinear extension.

As we discussed in class, a polynomial commitment has two basic operations. First, there is an operation Commit to generate a commitment to a polynomial. (The commitment protocol can be interactive, but Quokka has a non-interactive commitment phase.) Given this commitment, a prover and verifier can engage in an opening protocol Open to allow the verifier to learn the evaluation of the committed polynomial at a point of its choice.

The basic idea of Quokka is to exploit a natural way to view evaluating a multilinear extension as a matrix-vector (actually, a vector-matrix-vector) operation. More concretely, take the evaluations $p(i) = p_i$

$\mathsf{Quokka.Pgen}(\ell)$:

$G_1, \ldots, G_m \leftarrow_{\$} \mathcal{G}$

Return $(G_1, \ldots, G_m)$

$\mathsf{Quokka.Commit}(\mathsf{pp}, P)$:

$M \leftarrow \mathsf{matrix}(P)$

For $i \in [m]$:

$\quad c_i \leftarrow \sum_{j=1}^{m} M_{i,j} G_j$

Return $c_1, \ldots, c_m$

$\mathsf{Quokka.Open[Prover]}(\mathsf{pp}, P, \vec{C})$		$\mathsf{Quokka.Open[Verifier]}(\mathsf{pp}, \vec{C}, \vec{r})$
$C_1, \ldots, C_m \leftarrow \vec{C}$		$C_1, \ldots, C_m \leftarrow \vec{C}$ $\vec{b} \leftarrow [\chi_k(\vec{r}_0, \ldots, \vec{r}_{\ell/2-1})]_{k \in [m]}$ $C' \leftarrow \sum_{k=1}^{m} \vec{b}_k C_k$
	$\xleftarrow{\quad \vec{r}, C' \quad}$	
$M \leftarrow \mathsf{matrix}(P)$ $\vec{c} \leftarrow \sum_{k=1}^{m} \chi_k(\vec{r}_0, \ldots, \vec{r}_{\ell/2-1}) M_k$		
	$\xrightarrow{\quad \vec{c} \quad}$	
		Check $C' = \sum_{i=1}^{m} \vec{c}_i G_i$ $\vec{a} \leftarrow [\chi_j(\vec{r}_{\ell/2}, \ldots, \vec{r}_{\ell-1})]_{j \in [m]}$ Return $\langle \vec{c}, \vec{a} \rangle$

Figure 2: The Quokka polynomial commitment protocol. P is a multilinear polynomial in $n = m^2 = 2^\ell$ variables, and $\vec{r}$ is a vector of ℓ variables (ℓ is even). The procedure matrix takes a multilinear polynomial P and returns a matrix M of size $m \times m$ where $M_{j,k} = P_{jm+k}$ and P_{jm+k} is the coefficient of the monomial χ_{jm+k} in the evaluation form of P.

and arrange them in a matrix (call it M) in row-major fashion, so that $M_{j,k} = p_{jm+k}$. (In words, we are setting the first row of M to be the evaluations of p at $[0, \ldots, m-1]$, the second row to be the next m, and so on.)

Now let $\vec{r} \in \mathbb{F}^\ell$ be the point at which we want to evaluate $\tilde{p}$. We know our closed-form for $\tilde{p}$ is

$$\sum_{i \in \{0,1\}^\ell} p(i_0, \ldots, i_{\ell-1}) \chi_i(\vec{r}_0, \ldots, \vec{r}_{\ell-1}) \ .$$

Rewriting χ_i as a product of two basis polynomials on $\ell/2$ variables, we can write the above as

$$\sum_{i \in \{0,1\}^\ell} p(i_0, \ldots, i_{\ell-1}) \chi_{i_0, \ldots, i_{\frac{\ell}{2}-1}}(\vec{r}_0, \ldots, \vec{r}_{\frac{\ell}{2}-1}) \chi_{i_{\frac{\ell}{2}}, \ldots, i_{\ell-1}}(\vec{r}_{\frac{\ell}{2}}, \ldots, \vec{r}_{\ell-1}) \ .$$

It may not look like we've done much, but splitting up the basis polynomials lets us observe an interesting property of the summation: its first m terms—which, recall, is the contribution to the overall evaluation of $\tilde{p}$ of the p_i's appearing in the first row of the matrix M—are all multiplied by the exact same quantity $\chi_{i_{\frac{\ell}{2}}, \ldots, i_\ell}(\vec{r}_{\frac{\ell}{2}}, \ldots, \vec{r}_{\ell-1})$. This slightly counterintuitive fact is because the top half of the bits of i do not change for any of the p_is in the first row of M—in particular the bits $i_{\frac{\ell}{2}}, \ldots, i_{\ell-1}$ are zero for the whole row. (Recall we are using little-endian bit order, so i_0 is the least significant bit of i.) To clarify the next steps we will split the index i up into two halves: the low-order bits we will call k and the high-order bits we will call j (so $i = jm + k$ and j, k are each $\ell/2$ bits). Plugging j, k into the sum and rewriting a bit gives us

$$\sum_{j \in \{0,1\}^{\ell/2}} \sum_{k \in \{0,1\}^{\ell/2}} p(jm+k) \chi_k(\vec{r}_0, \ldots, \vec{r}_{\frac{\ell}{2}-1}) \chi_j(\vec{r}_{\frac{\ell}{2}}, \ldots, \vec{r}_{\ell-1}) \ .$$

Since the χ_j terms do not depend on the inner index of summation k, we can pull them out and rewrite as

$$\sum_{j \in \{0,1\}^{\ell/2}} \chi_j(\vec{r}_{\frac{\ell}{2}}, \ldots, \vec{r}_{\ell-1}) \sum_{k \in \{0,1\}^{\ell/2}} p(jm+k) \chi_k(\vec{r}_0, \ldots, \vec{r}_{\frac{\ell}{2}-1}) \ .$$

Again, it may not look like we've accomplished that much, but rewriting it this way lets us recognize some algebraic structure: for a fixed j, the inner sum over k is exactly the jth entry of a vector-matrix product. To see this, first define the vector $\vec{b}$ of length m where for all $k \in [m]$, $\vec{b}_k = \chi_k(r_0, \ldots, r_{\ell/2-1})$. Then we claim the jth entry of $\vec{b}^\top M$ is exactly

$$\sum_{k \in \{0,1\}^{\ell/2}} p(jm+k) \chi_k(\vec{r}_0, \ldots, \vec{r}_{\frac{\ell}{2}-1}) \ .$$

Now, this is interesting—we can express computing the outer sum over j (and thus finishing the evaluation of $\tilde{p}(\vec{r})$) as an inner-product of the vector $\vec{b}^\top M$ and an analogous vector of the $\ell/2$-variate basis polynomials evaluated at $r_{\ell/2}, \ldots, r_{\ell-1}$. Call such a vector $\vec{a}$. (Note, as above, formally $\vec{a}_j = \chi_j(r_{\ell/2}, \ldots, r_{\ell-1})$ for $j \in [m]$.) Then we can express the entire sum, and therefore also the evaluation we want to compute, as the vector-matrix-vector product $\vec{b}^\top M \vec{a}$.

Quokka **protocols.** The Quokka commitment and opening both use a set of generators of a group $\mathcal{G}$ where the discrete logarithm problem is hard. In your project this is the elliptic curve P-256. The commitment algorithm Commit simply commits to each row of the matrix M—built as described above—using a multi-scalar multiplication where the scalars are the row values and the points are the generators. A note on efficiency:

you should not need to explicitly materialize the matrix M; instead you can use clever indexing to view the evaluation table as the matrix.

Given the row commitments, the interactive opening protocol Open works by first having the verifier use the additive homomorphism of the row commitments to compute a *commitment to* the vector $\vec{b}^\top M$ for its chosen point $\vec{r}$. Then, the verifier sends the evaluation point $\vec{r}$ and its derived commitment to the prover. The prover commits the vector $\vec{b}^\top M$ in the clear and sends it to the verifier. The verifier checks that the prover's claimed result R is indeed a set of scalars that, when multiplied by the generators, gives C'. It then computes $\vec{a}$ itself and finishes by computing the inner product between R and $\vec{a}$. As we argued above, if both parties have followed the protocol, this inner product is exactly $P(\vec{r})$.

Intuitively, this protocol is secure because C' is a binding commitment to the vector $\vec{b}^\top M$, so a malicious prover cannot send an incorrect $\vec{c}$ that causes the verifier's last check to pass. By correctness, as long as the vector $\vec{c}$ is correct, the output must be correct as well.

Quokka needs one commitment consisting of a single group element for each row of the matrix M; thus there are m group elements total. The communication sent between the prover and verifier during opening is proportional to m—the evaluation point $\vec{r}$ is only ℓ field elements, the derived commitment C' is one group element, and the vector $\vec{c}$ is m field elements. Finally, and perhaps most surprisingly, the verifier's *computation* is proportional to m. Computing the vectors $\vec{b}$ and $\vec{a}$ takes $O(m)$ field operations, computing the derived commitment C' and checking the vector $\vec{c}$ takes (even with naive algorithms) $O(m)$ scalar multiplications and point additions, and the final inner product is also $O(m)$.

3 The Delphian Succinct Argument of Knowledge

Your final task in this project will be to implement the Delphian succinct argument of knowledge for R1CS satisfiability. We give the prover and verifier algorithms for the Delphian protocol in Figure 3. A protocol very similar to Delphian is discussed in Section 8.4 of Thaler [1]. Delphian is an example of the polynomial IOP to argument compilation we discussed in lecture. The key idea underlying Delphian is that we can translate a single statement about R1CS satisfiability into a small number (concretely, four) of statements about the sum of a multivariate polynomial over the hypercube, as well as a few auxiliary statements. Once we do this, we can use the sumcheck protocol to reduce these statements into claims about multilinear polynomial evaluations at random points; finally, we can use Quokka to verify these claims.

Reducing R1CS to sumcheck. As above, call the R1CS matrices A, B, C and assume they are $n \times n$ matrices over $\mathbb{F}$. Let the public input be a vector $x \in \mathbb{F}^{n/2}$ and the witness be a vector $w \in \mathbb{F}^{n/2}$. Let $z = (x||w)$. To start, we define functions f_A, f_B, f_C that encode the entries of the R1CS matrices. Each function is from $\{0,1\}^{\log n} \times \{0,1\}^{\log n} \to \mathbb{F}$. The function f_A is defined as $f_A(i,j) = A_{i,j}$ and likewise for B and C. Define the function f_{res} as

$$f_{\text{res}}(X) = \left(\sum_{b \in \{0,1\}^{\log n}} f_A(X,b)z(b)\right)\left(\sum_{b \in \{0,1\}^{\log n}} f_B(X,b)z(b)\right) - \left(\sum_{b \in \{0,1\}^{\log n}} f_C(X,b)z(b)\right).$$

It is not too difficult to see that if $f_{\text{res}}(X) = 0$ for all $X \in \{0,1\}^{\log n}$, then the R1CS instance is satisfied. In words, this is because $f_{\text{res}}(i)$ is computing the ith entry of the vector $Az \circ Bz - Cz$. If each entry is zero, this means $Az \circ Bz = Cz$.

Delphian first phase. The Delphian prover's job is to convince the verifier that $f_{\text{res}}(X) = 0$ for all $X \in \{0,1\}^{\log n}$. The verifier could check f_{res} itself, but this would not be efficient in verifier time or

$\mathsf{Prover}(A, B, C, x, w)$		$\mathsf{Verifier}(A, B, C, x)$
compute multilinear extension $\tilde{w}$ of w commit to $\tilde{w}$ using $\mathsf{PC.Commit}(\tilde{w}) \to c_{\tilde{w}}$		
	$\xrightarrow{c_{\tilde{w}}}$	
		$r \xleftarrow{\$} \mathbb{F}^{\log n}$
	$\xleftarrow{r}$	
compute $h(x) = \widetilde{\mathsf{eq}}(r, x) g_{\mathrm{res}}(x)$		
	sumcheck reduce claim $\sum_{x \in \{0,1\}^{\log n}} h(x) = 0$ $\xleftrightarrow[\text{to claim } r' = (r'_1, \ldots, r'_{\log n}),\, H']{}$	
for $M \in \{A, B, C\}$, $p_M(x) = \tilde{f}_M(r', x)\tilde{z}(x)$ $v_M = \sum_{x \in \{0,1\}^{\log n}} p_M(x)$		$e \leftarrow \widetilde{\mathsf{eq}}(r, r')$; $G' \leftarrow H'/e$ [Note G' is claimed to be $g_{\mathrm{res}}(r')$]
	$\xrightarrow{v_A, v_B, v_C}$	
		Check $v_A * v_B - v_C = G'$
..... Complete for $M \in \{A, B, C\}$:		
	sumcheck reduce claim $\sum_{x \in \{0,1\}^{\log n}} p_M(x) = v_M$ $\xleftrightarrow[\text{to claim } r''_M = (r''_{M,1}, \ldots, r''_{M,\log n}),\, H'']{}$	
$w_M \leftarrow \tilde{w}(r''_M[:-1])$		[Note H'' is claimed to be $p_M(r''_M) := \tilde{f}_M(r', r''_M)\tilde{z}(r''_M)$]
	$\xrightarrow{w_M}$	
	$\xleftrightarrow{\mathsf{PC.Open}(c_{\tilde{w}}, r''_M[:-1], w_M)}$	
		$z_M = (1 - r''_M[-1])\tilde{x}(r''_M[:-1]) + r''_M[-1] w_M$ $e_M = \tilde{f}_M(r', r''_M) z_M$ Check $H'' = e_M$
..... Final check		
		If any subprotocols output $\perp$ or checks fail, output $\perp$, else True

Figure 3: The Delphian succinct argument of knowledge. The R1CS matrices A, B, C are $n \times n$ and $|x| = |w| = n/2$.

communication. Instead, the Delphian prover does the following. First, it transforms f_{res} into a low-degree multivariate polynomial g_{res}, by taking the multilinear extension of each component function.

$$g_{\text{res}}(X) = \left(\sum_{b \in \{0,1\}^{\log n}} \tilde{f}_A(X, b)\tilde{z}(b) \right) \left(\sum_{b \in \{0,1\}^{\log n}} \tilde{f}_B(X, b)\tilde{z}(b) \right) - \left(\sum_{b \in \{0,1\}^{\log n}} \tilde{f}_C(X, b)\tilde{z}(b) \right)$$

The polynomial g_{res} is a log-n-variate polynomial with maximal degree two. By definition, it agrees with f_{res} on all inputs in $\{0,1\}^{\log n}$. Thus, if $g_{\text{res}}(X) = 0$ for all $X \in \{0,1\}^{\log n}$, then $f_{\text{res}}(X) = 0$ for all $X \in \{0,1\}^{\log n}$ and the R1CS instance is satisfied. It may not seem like we've gained much, because to check $g_{\text{res}}(X) = 0$ for all $X \in \{0,1\}^{\log n}$ still requires n evaluations.

However, the insight we can bring to bear is that, using some randomness from the verifier, the prover can transform the statement $\forall X \in \{0,1\}^{\log n}, g_{\text{res}}(X) = 0$ into a sumcheck-like statement for some h that is related to g_{res}. This statement can be checked efficiently using the sumcheck protocol. The construction of h relies on the following lemma, which theory-track students will prove later in the semester.

Lemma 3.1. *Let f_{res} and g_{res} be as defined above. Let* $\mathsf{eq} : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}$ *be the function that takes two n-bit vectors and returns 1 if they are component-wise equal, and 0 otherwise. Let* $\widetilde{\mathsf{eq}}$ *be the multilinear extension of* eq. *Then if $\exists y \in \{0,1\}^n$ such that $f_{\text{res}}(y) \neq 0$, then*

$$\Pr_{r \leftarrow \$ \mathbb{F}^n} \left[\sum_{x \in \{0,1\}^n} \widetilde{\mathsf{eq}}(r, x) g_{\text{res}}(x) = 0 \right] \leq \frac{n}{|\mathbb{F}|} .$$

Thus, the first phase of Delphian boils down to proving that the hypercube sum of $h(X) = \widetilde{\mathsf{eq}}(r, X) g_{\text{res}}(X)$ is zero. (Note that h is also log-n-variate and has maximal degree three.) The prover begins by committing to the extension of its witness w using Quokka,then sending the commitment to the verifier. Then, the verifier sends a random r to define h, then the prover and verifier perform a sumcheck to reduce the claim $\sum_{X \in \{0,1\}^{\log n}} h(X) = 0$ into a claim that the evaluation of h at a random point r' is H'. (Note the claim $h(r') = H'$ is, by definition of h, equal to the claim $\widetilde{\mathsf{eq}}(r, r') g_{\text{res}}(r') = H'$.)

Delphian second phase. Checking this claim efficiently is, roughly, the purpose of the rest of the Delphian protocol. The verifier can efficiently evaluate $\widetilde{\mathsf{eq}}(r, X)$—the extension of the equality function has a small explicit representation. Call this value e. (In class we defined the multilinear Lagrange basis polynomial χ_i—this is exactly $\widetilde{\mathsf{eq}}(i, X)$.)

The polynomial g_{res} is still too large for the verifier to evaluate directly. By construction, though, we can decompose the claim about g_{res} into three claims about the evaluations of the three component functions, one each for A, B, and C:

$$g_{\text{res}}(r') = \left(\sum_{b \in \{0,1\}^{\log n}} \tilde{f}_A(r', b)\tilde{z}(b) \right) \left(\sum_{b \in \{0,1\}^{\log n}} \tilde{f}_B(r', b)\tilde{z}(b) \right) - \left(\sum_{b \in \{0,1\}^{\log n}} \tilde{f}_C(r', b)\tilde{z}(b) \right) .$$

Let $p_M(x) = \tilde{f}_M(r', x)\tilde{z}(x)$, so $g_{\text{res}}(r') = (\sum_b p_A(b))(\sum_b p_B(b)) - (\sum_b p_C(b))$. Fortunately for us, each of the sums in the parentheses is already in the form of a sumcheck statement! Thus, in the second phase of Delphian, the prover sends the verifier the claimed evaluations v_A, v_B, v_C of each of the three sums, then does three sumchecks for the claims $\sum_b p_M(b) = v_M$ for each of A, B, and C. (A useful exercise for checking your understanding at this point is to ask why the verifier must check that $v_A v_B - v_C = G'$.)

These sumchecks let us reduce to a claim about the evaluation of p_M at a random point r''. By construction, $p_M(r''_M) = \tilde{f}_M(r', r''_M)\tilde{z}(r''_M)$. Since $\tilde{f}_M$ is public, the verifier can compute it. This computation, however, is *not* efficient—$\tilde{f}_M$ has no special structure, so evaluating it naively takes n^2 field operations in the worst case. (As a practical matter, R1CS matrices for most real computations are very sparse—all but a small fraction of the evaluations of $\tilde{f}_M$ are zero. Thus, $\tilde{f}_M(r', r''_M)$ takes more like $O(n)$ field operations. Still, this check means Delphian is only succinct in communication and not in verifier time.)

The last thing we have to do is evaluate $\tilde{z}(r''_M)$. This is where we will use the committed witness from the first step: since $\tilde{z}$ is an extension of z, and z was constructed as $x\|w$, we can compute $\tilde{z}(r''_M)$ by checking a claimed evaluation of $\tilde{w}$ using the polynomial commitment's opening protocol and having the verifier evaluate the extension $\widetilde{x}$ of the input itself.

4 Submission Instructions and Grading Criteria

To submit your project, first run the command `zip submission.zip -r p1/src/ p2/src -x '*/target/*'` in the root of p2. Then, submit the resulting `submission.zip` file to autograder.io.

Autograder.io will run our test suite using your submitted code and compute your grade. Your grade will be the percentage of tests that pass. Any modifications you make to the `tests/` directory will not impact how your code is graded. (Feel free to add your own tests locally; this can be a good way to test your own understanding.)

Note that doing something "fishy" to try to get a better grade—for example, overwriting any equality methods—will result in failing the assignment.

References

[1] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*. Foundations and Trends in Privacy and Security, 2022.